

SoGoodFeed.sol

SoGoodFeed.sol

Refactor Summary:

Contract name: SoGoodFeed

Purpose: On-chain registry for social utility posts + AI-tipped token logs

All variables, structs, events, roles, and comments refactored to match:

Sotility ecosystem

SoGood social layer

Ready for integration with your AI oracle or Chainlink Automation

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/Counters.sol";

/**
 * @title SoGoodFeed
 * @dev AI-verified social tipping feed
 for the SoGood Utility Governance
 (SUG) token.
 */
contract SoGoodFeed is AccessControl
{
    using Counters for Counters.Counter;

    bytes32 public constant
VERIFIER_ROLE =
keccak256("VERIFIER_ROLE");

    Counters.Counter private _postIds;

    struct Post {
        uint256 id;
        address author;
        string contentHash;
        string metadata; // optional: type,
category, or comment
        uint256 timestamp;
        uint256 totalTips;
    }

    mapping(uint256 => Post) public
posts;
    mapping(uint256 =>
mapping(address => uint256)) public
tipsByUser;

    event PostSubmitted(uint256 indexed
id, address indexed author, string
contentHash, string metadata);
    event Tipped(uint256 indexed
postId, address indexed tipper, uint256
amount);

    constructor() {

_grantRole(DEFAULT_ADMIN_ROLE,
msg.sender);
    }

    function submitPost(string calldata
contentHash, string calldata metadata)
external {
        _postIds.increment();
        uint256 postId = _postIds.current();

        posts[postId] = Post({
            id: postId,
            author: msg.sender,
            contentHash: contentHash,
            metadata: metadata,
            timestamp: block.timestamp,
            totalTips: 0
        });

        emit PostSubmitted(postId,
msg.sender, contentHash, metadata);
    }

    function recordTip(uint256 postId,
address tipper, uint256 amount)
external onlyRole(VERIFIER_ROLE) {
        require(posts[postId].author !=
address(0), "Invalid post");
        tipsByUser[postId][tipper] +=
amount;
        posts[postId].totalTips += amount;
        emit Tipped(postId, tipper, amount);
    }

    function getPost(uint256 postId)
external view returns (Post memory) {
        return posts[postId];
    }

    function totalPosts() external view
returns (uint256) {
        return _postIds.current();
    }
}
```

